

SQL Structured Query Language.

- Database is a collection of data in a format that can be easily accessed (digital).
- A software application used to manage our DB is called DBMS (Database Management System)

Types of database.

Relational

- Data stored in tables
- Eg:- MySQL, PostgreSQL, ORACLE SQL.

non-Relational (NoSQL)

Eg:- MongoDB.

we use SQL to work with relational DBMS.

What is SQL?

→ SQL is a programming language used to interact with Relational database.

it is used to perform CRUD operation.

Create
Read
Update
Delete.

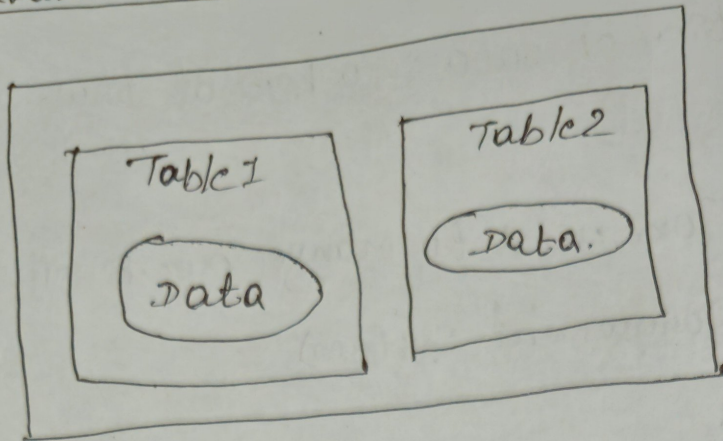
SEQUEL

Structured
English
Query
Language

SQL

Structured
Query
Language.

Database Structure.



Creating First database

CREATE DATABASE db-name;

DROP DATABASE db-name;

Creating table

use db-name;

CREATE TABLE table-name (

column-name1 datatype constraint,

column-name2 datatype constraint,

column-name3 datatype constraint,

);

Type of SQL command.

- DDL (Data Definition language): create, alter, rename, truncate & drop.
- DQL (Data Query language): select
- DML (Data Manipulation language): insert, update & delete.
- DCL (Data Control language): grant & Revoke permission to users.
- TCL (Transaction Control language): Start transaction, commit, roll back &.

Database Related queries.

CREATE DATABASE db.name;

CREATE DATABASE IF NOT EXISTS db.name;

CREATE DATABASE IF NOT EXISTS Collage;

DROP DATABASE db.name;

DROP DATABASE IF EXISTS db.name;

Show database;

Show tables;

Insert

INSERT INTO table-name
(colname1, colname2)

values

(col1-v1, col2-v1);

(col1-v2, col2-v2);

Keys

Primary key

- it is a column (or set of columns) in a table that uniquely identifies each row. (a unique id). There is only 1 PK & should be NOT NULL.

Foreign key

- A Foreign key is a column (or set of columns) in a table that refers to the primary key of another table. There can be multiple FKs.

FKs can have duplicate & null values.

Constraints

- SQL constraints are used to specify rules for data in a table.

NOT NULL columns cannot have a Null value Eg:- col1 int NOT NULL

UNIQUE all values in column are different Eg:- col2 int UNIQUE

- PRIMARY key makes a column unique & not null but used only for

Foreign key

- Prevents

Eg: CR

DEFAULT

- Set the

Eg:

CHECK

- it c

Foreign key.

- prevent actions that destroy links between tables.

Eg: CREATE TABLE temp1

custid int,

FOREIGN KEY (custid) reference customer(id).

);

DEFAULT

- set the default value of a column.

Eg:- Salary INT DEFAULT 25000

CHECK

- it can limit the values allowed in a column.

Eg:- CREATE TABLE city (

id INT PRIMARY KEY,

city VARCHAR(50),

age INT,

CONSTRAINT age-check CHECK

(age >= 18 AND city = "Delhi?")

);

CREATE TABLE NewTab (

age INT CHECK (age >= 18)

);

Select in detail

- used to select any data from the database.

Basic Syntax

- `SELECT col1, col2 FROM table-name;`

To Select All

- `SELECT * FROM table-name;`

* Where clause

- To define some conditions.

`SELECT col1, col2 FROM table-name;
WHERE conditions;`

Eg:- `SELECT * FROM Student WHERE Marks > 80;`

`SELECT * FROM Student WHERE City = "Mumbai";`

* Using operators in WHERE clause.

- * Arithmetic operator: + (addition), - (subtraction), * (multiplication), / (division), % (modulus)
- * Comparison operators: = (equal to), != (not equal to), >, >=, <, <=
- * Logical operators: AND, OR, NOT, IN, BETWEEN, ALL, LIKE, ANY
- * Bitwise operator: & (Bitwise AND), | (Bitwise OR).

operators.

- AND (to check for both conditions to be true).
SELECT * FROM student WHERE marks > 80 AND city = "Mumbai";
- OR (to check for one of the condition is true).
SELECT * FROM student WHERE marks > 90 OR city = "Mumbai";
- Between (select for a given range)
SELECT * FROM student WHERE marks BETWEEN 80 AND 90;
- IN (matches any value in the list).
SELECT * FROM student WHERE city IN ("Delhi", "Mumbai");
- NOT (to negate the given condition)
SELECT * FROM student WHERE city NOT IN ("Delhi", "Mumbai");

Limit clause

sets an upper limit on number of (tuples) row to be returned.

SELECT * FROM student LIMIT 3;

Syntax:-

SELECT col1, col2 FROM table name
LIMIT number;

* Order By clause

To sort in ascending (ASC) or descending order (DESC)

```
SELECT * FROM Student  
ORDER BY city ASC;
```

Syntax:-
SELECT col1, col2 FROM table-name
ORDER BY col-name(s) ASC;

* Aggregate Function.

Aggregate Function perform a calculation on a set of values, and return a ~~set~~ single value.

- COUNT()
- MAX()
- MIN()
- SUM()
- AVG()

Eg:- SELECT max(marks) FROM Student;

* GROUP By clause

Groups rows that have the same values into summary row. It collects data from multiple records and groups the result into one or more columns.

* Generally we use group with some aggregation function

count num.
SELECT
FROM
GROUP

* Having clause
similar to
used when

count num.
Cross join

Eg:- SE
Fr
G
H

General on

SELECT
FROM
WHERE
GROUP
HAVIN
ORDER

Count number of students in each city.

SELECT city, count(name)

FROM student

GROUP BY city;

✓

* Having clause

Similar to where i.e. applies some condition on rows.

Used when we want to apply any condition after grouping.

Count number of students in each city where max marks cross 90.

Eg:- SELECT count(name), city

FROM student

GROUP BY city

HAVING max(marks) > 90;

General order For writing a clause

SELECT column(s)

FROM table-name

WHERE condition

GROUP BY column(s)

HAVING condition

ORDER BY column(s) ASC;

Table related queries.

update (to update existing rows)

Syntax:-

UPDATE table-name

SET col1 = val1, col2 = val2

WHERE condition;

Eg update students

SET grade = "O"

WHERE grade = "A";

Query to change safe mode.

SET SQL_SAFE_UPDATES = 0;

DELETE (to delete existing rows).

Syntax:-

DELETE FROM table-name

WHERE condition;

Eg:- DELETE FROM students

WHERE marks < 33;

Foreign key.

Eg:- FOREIGN KEY (dept-id) REFERENCES dept (id).

Primary key.

Cascading For F

on ~~update~~ ^{delete} cascade

when we create the referencing row is delete

update cascade

when we create referencing row referenced row is delete

Eg:- CREATE

id

col

for

o

o

);

Alter C

ADD col

ALTER T

ADD col

DROP C

ALTER T

DROP col

Cascading For FK

on ~~delete~~ cascade

when we create a Foreign key using this option, it deletes the referencing rows in the child table. when the referenced row is deleted in the parent table which has a primary key.

on ~~update~~ cascade.

when we create a Foreign key using UPDATE CASCADE the referencing rows are updated in the child table when the referenced row is updated in the parent table which has a primary key.

Ex: CREATE TABLE Student {

id INT PRIMARY KEY,

course ID INT,

FOREIGN KEY (course ID) REFERENCE course (id).

ON DELETE CASCADE

ON UPDATE CASCADE

};

Alter (to change the schema)

• ADD column

ALTER TABLE table-name

ADD column-name datatype constraint;

key.

• DROP column

ALTER TABLE table-name

DROP column-name;

- RENAME Table
ALTER TABLE table-name
RENAME TO new-table-name;
- CHANGE column (rename)
ALTER TABLE table-name
CHANGE COLUMN old-name new-name new-datatype new-constraint;
- MODIFY column (modify datatype / constraint);
ALTER TABLE table-name
MODIFY col-name new-datatype new-constraint;
- TRUNCATE TABLE table-name;

UPDATE Student
SET grade = "O"
WHERE grade = "A";

* JOIN in SQL

- JOIN is used to combine rows from two or more tables, based on a related column between them.

Type of JOINs

- inner JOIN
- Left JOIN
- Right JOIN
- Full JOIN

Inner JOIN

- Return rows

Syntax

SELECT
FROM
~~TABLE~~
INNER
JOIN
ON to

Left JOIN

Returns
Records

Syntax:

SELECT
FROM
LEFT
JOIN
ON

Right

Returns
Records

Syntax

SELECT
FROM
RIGHT
JOIN
ON

Inner Join

- Return records that have matching values in both tables.

Syntax

```
SELECT column(s)
FROM tableA
INNER JOIN
INNER JOIN tableB
ON tableA.column_name = tableB.col_name;
```

Left Join

Returns all records from the left table, and the matched records from the right table.

Syntax:-

```
SELECT column(s)
FROM tableA
LEFT JOIN tableB
ON tableA.col_name = tableB.col_name;
```

Right Join

Returns all records from the right table, and the matched records from the left table.

Syntax

```
SELECT column(s)
FROM tableA
RIGHT JOIN tableB
ON tableA.col_name = tableB.col_name;
```


Full Join

Return all records when there is a match in either left or right table.

Syntax:-

```
SELECT * FROM Student as a
LEFT JOIN course as b
ON a.id = b.id
UNION
SELECT * FROM Student as a
RIGHT JOIN course as b
ON a.id = b.id;
```

• Left Exclusive Join / Right Exclusive Join.

SELECT * FROM Student as a LEFT JOIN course as b ON a.id = b.id WHERE b.id IS NULL;	SELECT * FROM Student as a LEFT RIGHT JOIN course as b ON a.id = b.id WHERE a.id IS NULL;
---	--

Self Join.

it is regular join but the table is joined with itself.

Syntax:- SELECT columns)

```
FROM table as a
JOIN table as b
ON a.col_name = b.col_name;
```


left or

UNION

- it is used to combine the result-set of two or more SELECT statements.
- Gives UNIQUE records.
- it remove duplicate records.

To use it:

- every SELECT should have same no. of columns
- Columns must have similar data types.
- Columns in every SELECT should be in same order.

Syntax:-

```
SELECT column(s) FROM table A
UNION
SELECT column(s) FROM table B.
```

UNION ALL

- it allow duplicate records.

SOL Sub Queries.

- A subquery or inner query or a nested query is a query within another SQL query.
- it involves 2 select statements.

Syntax:-

```
SELECT column(s)
FROM table-name
WHERE col-name operator
(subquery);
```


SQL Sub Queries.

Example with SELECT.

Eg:- SELECT (SELECT MAX(marks) FROM student), name
FROM student;

MySQL Views.

A view is a virtual table based on the result-set of an SQL statement.

CREATE VIEW view1 AS

SELECT rollno, name FROM ~~table~~ student;

SELECT * FROM view1;

* A view always show up-to-date data. The database engine recreates the view, everytime a user queries it.